



Ingénierie et Intégration des Services Web

Dr Papa Samour DIOP
Enseignant chercheur en
Informatique

Module 1 :

Fondations de l'Architecture Client-Serveur et HTTP

Le Paradigme de Communication : Le Modèle Requête/Réponse

Définition

Le modèle **Client-Serveur** est un mode de communication réseau où les tâches sont réparties entre des fournisseurs de ressources ou de services, appelés **serveurs**, et des demandeurs de services, appelés **clients**.

- **Le Client** : C'est l'entité (souvent un navigateur comme Chrome ou une application mobile) qui initialise la communication en envoyant une unité de données appelée **Requête (Request)**.
- **Le Serveur** : C'est une machine distante, à l'écoute permanente, qui traite la requête, accède aux données nécessaires, et renvoie une **Réponse (Response)**.

L'Analogie Sénégalaise : "Le Client au Restaurant de Thieb"

- Imaginez que vous allez dans un restaurant réputé à Dakar (le **Serveur**).
- **Le Client (Vous)** : Vous arrivez et vous ne vous servez pas vous-même en cuisine. Vous passez une commande précise : "Un Thieboudienne Penda Mbaye, s'il vous plaît" (**La Requête**).
- **Le Serveur (Le restaurateur)** : Il reçoit votre commande, vérifie s'il reste du riz en cuisine (traitement logique/accès base de données), prépare l'assiette et vous l'apporte (**La Réponse**).
- **Règle d'or** : Dans le Web (HTTP), le serveur ne vous appellera jamais de lui-même pour vous proposer un Thieb si vous n'avez pas d'abord envoyé une requête. **C'est toujours le client qui initie l'échange.**

Communication Synchrone vs Asynchrone (Niveau Transport)

- Au niveau du transport des données, la manière dont le client et le serveur "s'attendent" définit la nature de la communication.

La Communication Synchrone

- **Définition** : Dans un mode synchrone, l'émetteur (le client) envoie une requête et **bloque son exécution** (il s'arrête) en attendant la réponse du récepteur (le serveur). Le flux de contrôle est lié au temps de réponse.
- **Analogie** : Un appel téléphonique classique. Si vous appelez la SENELEC pour un dépannage, vous restez l'oreille collée au téléphone. Vous ne pouvez rien faire d'autre de productif tant que l'agent ne vous a pas répondu ou n'a pas raccroché.
- **Inconvénient** : Si le serveur est lent (comme une connexion instable à Dakar lors d'un orage), le client "freeze" ou tourne dans le vide.

La Communication Asynchrone

- **Définition** : Dans un mode asynchrone, l'émetteur envoie sa requête et **reprend immédiatement son travail** sans attendre la réponse. Il sera notifié plus tard (via un mécanisme de rappel ou *callback*) quand la réponse sera prête.
- **Analogie** : Envoyer un message **WhatsApp** ou un **SMS**. Vous envoyez votre question : "Fii fan la gare TER bi nekk ?" (Où se trouve la gare du TER ?). Vous rangez votre téléphone, vous continuez à marcher, vous achetez des arachides. Quand votre ami répond, votre téléphone vibre. Vous n'avez pas perdu de temps à attendre sans bouger.
- **Avantage** : C'est ce qui permet aux applications modernes d'être fluides. Pendant qu'une image lourde charge en arrière-plan, vous pouvez continuer à scroller le texte.

Pourquoi est-ce crucial pour un Ingénieur Web ?

Comprendre cette distinction est la base de la performance :

- **Le protocole HTTP est fondamentalement synchrone** (une requête attend une réponse).
- Cependant, en utilisant des technologies comme **AJAX** ou les **Web Workers** en JavaScript, nous simulons de l'asynchronisme pour que l'utilisateur sénégalais, même avec une connexion 3G/4G capricieuse, n'ait pas l'impression que son application est "plantée".
- **Note** : Maîtriser ce paradigme, c'est comprendre que le Web n'est pas une conversation continue, mais une succession de "coups de fil" (requêtes) très rapides et structurés.
- Nous entrons maintenant dans le "moteur" du Web. Pour un ingénieur, comprendre **HTTP/1.1**, ce n'est pas juste connaître des abréviations, c'est comprendre les règles de politesse et de sécurité qui régissent l'échange d'informations sur Internet.

Anatomie du Protocole HTTP/1.1

- Le protocole HTTP (HyperText Transfer Protocol) est un protocole **textuel** et **sans état (stateless)**. Imaginez-le comme un formulaire standardisé que tout le monde doit remplir de la même manière pour être compris par le serveur.

Sémantique des Méthodes : Le "Verbe" de l'Action

- Chaque requête commence par un verbe qui indique l'intention du client. Mais attention, tous les verbes n'ont pas les mêmes conséquences sur le serveur (la base de données).

A. Méthodes Sûres (Safe Methods)

- **Définition** : Une méthode est dite **Sûre** si son exécution n'entraîne aucune modification de l'état de la ressource sur le serveur. Elle est en "lecture seule".
- **Exemple type** : GET, HEAD.
- **Analogie** : Vous allez au garage Petersen pour regarder le tableau des horaires des bus Dakar Dem Dikk. Le simple fait de **lire** l'affiche ne change pas l'heure du bus. Vous avez consulté l'information sans modifier la réalité.

B. Méthodes Idempotentes (Idempotency) et C. Le Danger : Pourquoi GET ne doit JAMAIS modifier la base de données ?

B. Méthodes Idempotentes (Idempotency)

- **Définition** : Une méthode est **Idempotente** si le fait de l'appeler une seule fois ou plusieurs fois successivement a exactement le même effet sur le serveur.
- **Exemple type** : PUT (remplacer), DELETE (supprimer), GET.
- **Analogie** : Vous envoyez un Wave à un marchand. Si l'opération de "fixer le prix à 5000 FCFA" est idempotente, alors même si vous cliquez 10 fois sur le bouton "Mettre à jour le prix", le prix restera bloqué à 5000 FCFA. L'état final est identique.
- **Contre-exemple (Non-Idempotent)** : POST. Si vous cliquez 10 fois sur "Envoyer 5000 FCFA", vous risquez de vider votre compte car chaque clic crée une nouvelle transaction.

C. Le Danger : Pourquoi GET ne doit JAMAIS modifier la base de données ?

- Imaginez qu'un développeur crée un lien : <https://mabanque.sn/virement?montant=10000&destinataire=Awa>.

Si ce lien utilise un GET pour effectuer le virement :

- **Mise en cache** : Le navigateur pourrait pré-charger le lien, déclenchant le virement sans votre accord.
- **Historique** : Le lien (et donc le montant/destinataire) reste dans l'historique du navigateur.
- **Bots** : Un robot d'indexation (Google Bot) qui parcourt le site pourrait "cliquer" sur le lien et vider les comptes par erreur.
- **Règle d'or** : GET est pour la consultation. POST/PUT/PATCH est pour l'action.

Étude des Headers (En-têtes) : Les "Coulisses" de la Requête

- Les headers sont des couples Clé: Valeur qui donnent des métadonnées sur la requête ou la réponse.

A. Négociation de Contenu (Content Negotiation)

- C'est le dialogue entre le client et le serveur pour choisir le "format" de la donnée.
- **Accept (Côté Client)** : Le client dit ce qu'il est capable de comprendre.
 - *Analogie* : "Je parle Wolof et Français, envoie-moi les infos dans une de ces langues."
- **Content-Type (Côté Serveur/Client)** : Indique le format réel du corps de la donnée (ex: application/json, text/html).
 - *Analogie* : C'est l'étiquette sur un colis qui dit "Attention, ceci est du verre" ou "Ceci est un document PDF".

B. Gestion du Cache : Économiser la Bande Passante (Crucial au Sénégal)

- Au Sénégal (en Afrique), où le pass internet peut coûter cher ou être lent, le cache est vital.
- **ETag (Entity Tag)** : C'est une "empreinte digitale" (un code unique) d'une ressource.
 - *Fonctionnement* : Le serveur envoie une image avec un ETag: "v1". La prochaine fois, le client demande : "Est-ce que l'image a changé depuis 'v1' ?". Si non, le serveur répond 304 Not Modified (Réponse très légère, pas besoin de renvoyer l'image).
- **If-Modified-Since** : Le client demande la ressource uniquement si elle a été modifiée après une date précise.
 - *Analogie* : Vous demandez au boutiquier du coin : "Est-ce que le pain est arrivé depuis 6h du matin ?". S'il répond "Non", vous n'avez pas besoin de rentrer dans la boutique pour vérifier, vous économisez de l'énergie (et du crédit data).

Résumé

Concept	Rôle	Analogie
GET	Lire l'info	Regarder le menu au restaurant.
POST	Créer l'info	Commander un plat (nouvelle commande).
Idempotence	Répétabilité	Éteindre une lampe déjà éteinte (rien ne change).
Accept	Préférence	"Je préfère que tu me parles en Wolof".
ETag	Vérification	Vérifier si le sceau d'une bouteille est intact.

Pour devenir un véritable ingénieur en services Web, il ne faut pas seulement connaître les définitions, il faut comprendre la "mécanique interne" de la donnée. Nous allons décortiquer la **Sérialisation** sous toutes ses coutures, en zoomant sur la structure technique et l'impact réel sur les réseaux sénégalais.

Sérialisation de données : L'Art d'emballer l'Information

Le Concept Fondamental : Pourquoi sérialiser ?

- En informatique, un "Objet" (par exemple, un profil utilisateur avec nom, âge et solde Wave) vit dans la mémoire vive (RAM) de l'ordinateur sous une forme binaire complexe que seul le langage utilisé (Java, Python, PHP) comprend.
- **Le problème** : On ne peut pas envoyer de la RAM brute à travers un câble sous-marin ou une antenne 4G.
- **La solution** : La **Sérialisation**. C'est l'action de "platir" cet objet complexe pour le transformer en une suite de caractères (du texte) que n'importe quelle machine peut lire.

Définition

- La **sérialisation** est le processus de conversion d'un objet (une structure de données en mémoire vive, comme une liste d'étudiants en Java ou Python) en un format de texte ou binaire qui peut être stocké ou transmis sur un réseau (via HTTP).
- L'objectif est que le récepteur puisse effectuer l'opération inverse, la **désérialisation**, pour reconstruire l'objet original, même s'il utilise un langage de programmation différent.

XML (Extensible Markup Language) : La Rigueur Administrative

- Le XML n'est pas juste du texte avec des balises ; c'est un format **auto-descriptif** et extrêmement **robuste**.

A. Analyse Technique de la Structure

- **Les Balises (Tags)** : Chaque donnée est enfermée dans une balise d'ouverture et une balise de fermeture (ex: <solde>5000</solde>). Cela rend le fichier "verbeux" : on écrit souvent plus de balises que de données réelles.
- **Le Prologue** : Un fichier XML commence souvent par <?xml version="1.0" encoding="UTF-8"?>, définissant les règles de lecture dès le départ.
- **Les Attributs** : On peut ajouter des métadonnées à l'intérieur d'une balise (ex: <devise type="CFA">).

B. La Puissance du Contrat : DTD et XSD

C'est ici que le XML gagne sa place dans les banques sénégalaises (BCEAO, SGBS) :

- **XSD (XML Schema Definition)** : C'est le "vigile" à l'entrée du serveur. Il définit un dictionnaire de règles strictes.
- **Validation** : Si le serveur attend un numéro de téléphone (<tel>) de 9 chiffres commençant par 77, 78, 70 ou 76, et que vous envoyez du texte, le processeur XML rejette la requête **avant même** que le code de l'application ne la touche. C'est une sécurité structurelle.

C. L'Analogie "Le Dossier de l'Administration Publique"

- Imaginez que vous déposez un dossier de passeport à la police :
- Chaque feuille doit être à sa place, chaque case remplie exactement comme demandé (C'est le **XSD**).
- C'est lourd, il y a beaucoup de paperasse autour de votre simple nom (C'est le côté **Verbeux**).
- Mais à la fin, il n'y a aucune ambiguïté : le dossier est complet et certifié.

JSON (JavaScript Object Notation) : La Fluidité Numérique

- Le JSON est né du besoin de rapidité du Web moderne. C'est le format de prédilection des APIs REST et des applications mobiles.

A. Analyse Technique de la Structure

- **Paires Clé/Valeur** : Contrairement au XML, on ne ferme pas la balise. On écrit "nom": "Diop".
- **Types de données natifs** : Le JSON comprend nativement les nombres, les booléens (true/false), les tableaux [] et les objets {}.
- **Légèreté** : En supprimant les balises de fermeture, on réduit la taille du message de **30% à 50%** par rapport au XML.

B. Pourquoi est-ce "Natif en JavaScript" ?

- En JavaScript, un texte JSON est **déjà** presque un objet vivant. Il suffit d'une commande (JSON.parse()) pour que le navigateur transforme instantanément le texte en une variable utilisable. C'est ce qui rend les interfaces (comme le tableau de bord d'Orange et moi ou Wave) si réactives.

C. L'Analogie "La Discussion au Marché Tilène"

- Le JSON, c'est comme négocier avec un marchand :
- On va droit au but : "Prix ? 500. Quantité ? 2."
- Pas de protocole administratif inutile.
- C'est rapide, efficace, et si vous avez une petite connexion (peu de "crédit"), c'est le format qui passera le mieux.

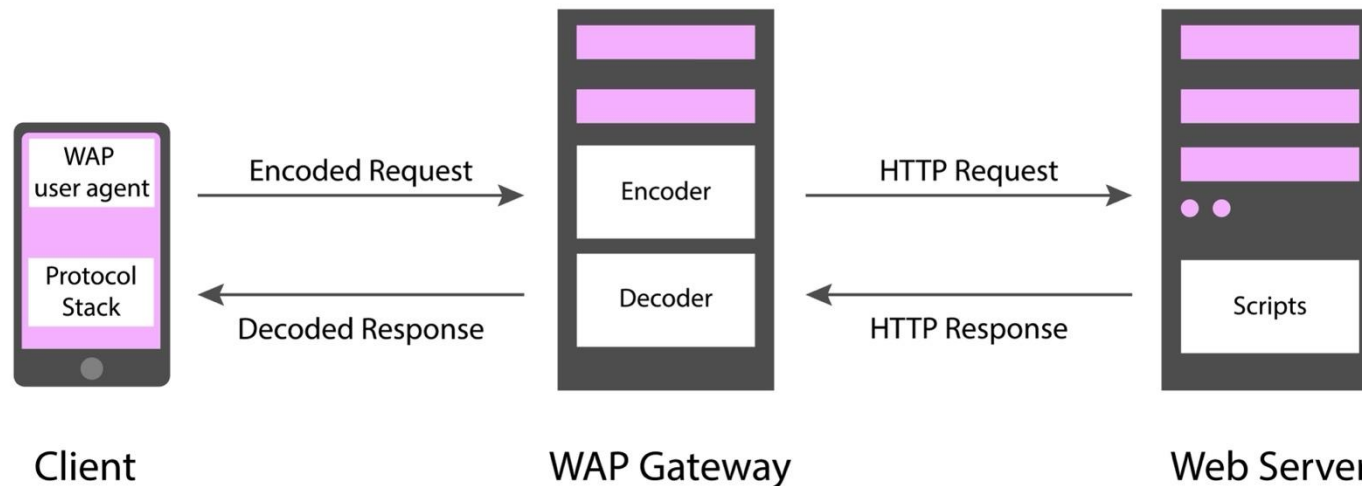
Le Grand Comparatif

Caractéristique	XML (Le "Sage")	JSON (Le "Rapide")
Poids des données	Lourd (consomme beaucoup de forfait)	Léger (économise la data)
Validation	Drastique (XSD/DTD)	Souple (souvent géré par le code)
Complexité	Élevée (nécessite des outils de parsing lourds)	Faible (simple lecture de texte)
Typage	Fort (Typage avec XSD)	Faible/Dynamique
Cas d'usage local	Virements interbancaires, Systèmes douaniers (GAINDE).	Apps de livraison (Yassir), Mobile Money, Réseaux Sociaux.

Wireless Application Protocol (WAP).

- C'est l'ancêtre de l'Internet mobile moderne. À l'époque des téléphones à touches (avant la 4G/5G), les mobiles n'avaient pas la puissance nécessaire pour lire le "vrai" Web (HTML lourd). Le WAP servait donc de traducteur.

Wireless Application Protocol



Les Composants de Wireless Application Protocol (WAP).

Le Client (Téléphone Mobile)

- C'est l'utilisateur. Comme les téléphones de l'époque étaient limités en mémoire et en batterie, ils utilisaient un **WAP User Agent** (un navigateur très simplifié). Au lieu d'envoyer des requêtes HTTP classiques, il utilise une **Protocol Stack** spécifique au WAP, beaucoup plus légère.



La Passerelle WAP (WAP Gateway)

- C'est l'élément central (le "traducteur"). Elle fait le pont entre le monde mobile (WAP) et le monde du Web classique (HTTP/TCP).
- **Encoder (Encodeur)** : Reçoit la requête du mobile, la traduit en une requête HTTP standard que le serveur Web peut comprendre.
- **Decoder (Décodeur)** : Reçoit la réponse HTML du Web, la comprime et la transforme en format binaire (WML - Wireless Markup Language) pour que le téléphone puisse l'afficher sans planter.



Le Serveur Web (Web Server)

- C'est un serveur classique, le même que celui utilisé par un ordinateur. Il héberge des **Scripts** et des données. Il ne sait pas qu'il parle à un téléphone ; il pense parler à un navigateur normal car la passerelle lui envoie des requêtes HTTP standard.

Le Flux de Communication (Étape par étape)

- **Requête Encodée (Encoded Request) :** Vous tapez une adresse sur votre ancien Nokia. Le téléphone envoie une requête compressée à la passerelle de votre opérateur (Orange, Free, etc.).
- **HTTP Request :** La passerelle décompresse la demande et l'envoie au serveur Web cible comme une requête Internet normale.
- **HTTP Response :** Le serveur répond en envoyant les données (souvent du HTML ou des scripts).
- **Réponse Décodée (Decoded Response) :** La passerelle reçoit ce contenu lourd, le transforme en données binaires très légères (optimisées pour les petits écrans et les processeurs lents) et les renvoie au téléphone.

Analogie Sénégalaise : "L'Interprète au Marché"

- Imaginez un touriste étranger (le **Serveur Web**) qui ne parle que l'Anglais et un commerçant au marché Sandaga (le **Client**) qui ne parle que le Wolof. Ils ne peuvent pas se comprendre directement.
- La **Passerelle WAP**, c'est l'interprète qui se tient entre les deux.
- Le commerçant fait une demande en Wolof (**Encoded Request**).
- L'interprète traduit en Anglais pour le touriste (**HTTP Request**).
- Le touriste répond avec une longue phrase complexe en Anglais (**HTTP Response**).
- L'interprète simplifie l'explication et la redonne en Wolof au commerçant (**Decoded Response**).

Résumé : Wireless Application Protocol

- L'architecture précédente illustre une époque où nos téléphones n'étaient pas des "smartphones".
- **WAP User Agent** : C'était le navigateur de votre ancien Nokia ou Samsung à touches.
- **WAP Gateway** : Comme Internet était trop "lourd" pour ces téléphones, cette passerelle servait de **traducteur**. Elle prenait le XML lourd du Web Server et le transformait en un format binaire ultra-compressé pour que le téléphone ne "plante" pas.
- **Aujourd'hui** : Grâce au **JSON**, nous n'avons plus besoin de cette passerelle complexe. Nos téléphones actuels sont assez puissants pour lire le JSON directement, même avec une connexion instable à l'intérieur du pays.

Conclusion de cette partie

- Un bon ingénieur ne choisit pas le format par mode.
- Si vous construisez un système pour le **Ministère des Finances**, le **XML** est votre allié pour sa rigueur.
- Si vous créez une startup pour la **livraison de repas à Dakar**, le **JSON** est indispensable pour sa légèreté et sa vitesse.

TP 1 : Exploration et Manipulation du Web Sénégalais

Partie 1 : Exploration avec cURL (Inspecter le Web local)

1. Qu'est-ce que cURL ?

- **cURL** est un outil en ligne de commande utilisé pour transférer des données. Contrairement à un navigateur qui affiche une page web jolie, cURL vous montre la "vérité nue" : les en-têtes (headers) et le code brut.

2. Exercice : Inspecter Seneweb.com

Ouvrez votre terminal (Invite de commande sur Windows ou Terminal sur Mac/Linux) et tapez la commande suivante :

- `curl -I https://www.seneweb.com`
- **Note** : L'option -I (majuscule) demande à cURL de n'afficher que les **headers** (l'en-tête), sans le contenu de la page.

Ce que vous allez voir (Explication détaillée) :

- **HTTP/2 200 OK** : Le serveur dit "J'ai bien reçu votre requête et tout va bien".
- **Content-Type: text/html** : Le serveur vous informe qu'il vous envoie une page web classique.
- **Server** : Indique souvent la technologie utilisée (Nginx ou Apache) cloudflare pour le site seneweb.com.
- **Cache-Control / ETag** : Regardez si Seneweb utilise ces headers pour économiser votre forfait internet à Dakar.

3. Analogie : Le "Gongo" du plat

- Utiliser cURL sans l'option -I, c'est recevoir tout le plat de Thieb. Utiliser -I, c'est juste sentir l'odeur du gongo pour savoir si le plat est bon avant de le manger.

Partie 2 : Manipulation avec Postman (L'API de Change XOF)

- L'objectif ici est de consommer une API réelle pour convertir l'Euro (€) ou le Dollar (\$) en **Franç CFA (XOF)**.

1. Préparation dans Postman

- Ouvrez **Postman**.
- Créez une nouvelle requête de type **GET**.
- Utilisez l'URL d'une API de change (ex: <https://api.exchangerate-api.com/v4/latest/EUR>).

2. Étape de Configuration

- **Méthode** : Sélectionnez GET (car nous voulons lire une information sans rien modifier sur le serveur).
- **Headers** : Cliquez sur l'onglet "Headers" pour voir ce que Postman envoie automatiquement (comme User-Agent ou Accept).

3. L'Analyse du résultat (JSON)

- Cliquez sur **Send**. Vous recevrez une réponse en format **JSON** (léger et facile à lire).

JSON

```
{  
  "base": "EUR",  
  "date": "2024-05-20",  
  "rates": {  
    "XOF": 655.95,  
    "USD": 1.08  
  }  
}
```

Travail à faire :

- Repérez la valeur du **XOF**.
- **Calculatrice sénégalaise** : Si vous recevez un virement de 100€ d'un parent en France, multipliez $100 * \text{rates.XOF}$.

Synthèse des outils du TP

Outil	Rôle dans ce TP	Pourquoi l'utiliser ?
cURL	Inspecteur de headers	Pour vérifier si <code>gouv.sn</code> OU <code>seneweb.com</code> gèrent bien le cache et la sécurité.
Postman	Client API	Pour tester la récupération de données financières (XOF) sans écrire de code.
JSON	Format de donnée	Pour recevoir le taux de change de manière légère, idéale pour nos réseaux mobiles.

Module 2 :

Web Services SOAP

Le Standard Industriel et Bancaire

Le Paradigme RPC (Remote Procedure Call) : L'Appel de Fonction à Distance

Définition

- Le **RPC** est un modèle de communication inter-processus qui permet à une application (le **Client**) d'exécuter une procédure ou une fonction sur une autre machine (le **Serveur**) sans se soucier des détails complexes du réseau (sockets, ports, protocoles de transport).
- Scientifiquement, le RPC repose sur le concept de **transparence de l'emplacement** : pour le programmeur, appeler une fonction située à 5 000 km ou une fonction située dans le même fichier de code doit être syntaxiquement identique.

La Mécanique Interne : Stub et Skeleton

Pour que cette "magie" opère, le RPC utilise deux agents invisibles :

- **Le Client Stub (La Marionnette)** : C'est un morceau de code généré automatiquement qui se fait passer pour la fonction réelle. Quand vous l'appellez, il intercepte vos données, les transforme en format XML (c'est le **Marshalling**) et les envoie sur le réseau.
- **Le Server Skeleton (Le Réceptionniste)** : Sur le serveur, ce composant reçoit le message, le traduit pour que le serveur le comprenne (c'est le **Unmarshalling**) et exécute la vraie logique métier.

L'Analogie Sénégalaise : "Le Guichet Unique de l'APIX"

- Imaginez que vous voulez créer une entreprise au Sénégal. Au lieu de courir à la mairie, puis au fisc, puis au tribunal, vous allez au **Guichet Unique de l'APIX** à Dakar.
- **Le Client (L'Entrepreneur)** : Vous arrivez au guichet avec vos documents. Pour vous, l'APIX est "la fonction" qui crée l'entreprise.
- **La Procédure Locale** : Vous tendez vos papiers à l'agent derrière la vitre. Vous ne voyez pas ce qu'il fait.
- **L'Exécution à Distance (RPC)** : L'agent de l'APIX utilise son logiciel pour appeler une fonction au **Ministère des Finances** et une autre au **Greffé du Tribunal**. Pour l'agent de l'APIX, c'est comme s'il tapait juste sur son clavier. En réalité, son ordinateur "appelle" des serveurs distants dans d'autres bâtiments pour valider votre dossier.
- **Le Résultat** : Quelques minutes plus tard, il vous dit "C'est bon, votre NINEA est prêt". L'action a été exécutée à distance, de manière transparente pour vous.

Pourquoi est-ce une Architecture "Orientée Action" ?

- Contrairement au Web classique (REST) où l'on cherche des "Ressources" (ex: /photos/1), SOAP et RPC se concentrent sur les **opérations**.

Type de besoin	Approche REST (Ressource)	Approche RPC/SOAP (Action)
Philosophie	"Donne-moi les infos du compte"	"Effectue le virement maintenant !"
Syntaxe	GET /comptes/123	<m:Virement><somme>50000</somme></m:Virement>
Analogie	Regarder le solde sur son téléphone.	Remplir un bordereau de retrait au guichet de la SGBS.

L'importance Cruciale : Sécurité et Rigueur

- Pourquoi la **BCEAO** (Banque Centrale) ou les douanes sénégalaises (**GAINDE**) utilisent-elles encore ce modèle rigide ?
- **Le Contrat est Loi** : En RPC/SOAP, on ne peut pas envoyer "n'importe quoi". Si la fonction de virement interbancaire attend un code banque (5 chiffres) et un montant (nombre positif), le système rejettera immédiatement toute requête qui ne respecte pas ce format avant même d'essayer de traiter l'argent.
- **La Gestion des Erreurs** : Si une erreur survient pendant l'appel (plus de réseau à Touba, serveur en maintenance à Dakar), le protocole RPC garantit que l'appelant reçoit une notification d'erreur précise (un **SOAP Fault**). On ne reste jamais dans l'incertitude : soit l'action est faite, soit elle a échoué.
- **Interopérabilité Totale** : Une application écrite en **C#** au siège d'une banque peut appeler une fonction RPC sur un serveur **Java** de la Sonatel sans aucun problème de compatibilité, car le "langage" de l'appel est standardisé.

Résumé

- Maîtriser le RPC, c'est comprendre que dans les systèmes critiques (Banque, État, Industrie), on ne se contente pas d'échanger des données, on **pilote des machines à distance** via un contrat de confiance indiscutable.
- Pour maîtriser l'ingénierie des services Web, il faut comprendre que **SOAP** n'est pas qu'un simple format d'échange, c'est un **protocole de communication complet** conçu pour la haute sécurité et la fiabilité industrielle.

L'Écosystème XML pour SOAP : La Rigueur du "Notaire Numérique"

- Dans un système comme celui de la **BCEAO** ou du **Trésor Public**, on ne peut pas se permettre d'interprétations floues. Chaque message doit être structuré, signé et validé selon des règles strictes.

L'Enveloppe SOAP : L'Unité de Transport Standardisée

- L'Enveloppe est la racine de tout document SOAP. C'est elle qui définit que "ceci est un message SOAP" et non un simple fichier XML.

A. Le Header (L'En-tête) : La "Tour de Contrôle"

- Le Header est facultatif mais crucial dans le monde industriel. Il contient des métadonnées qui ne sont pas liées à l'application elle-même, mais à la **gestion du message**.
- **Sécurité (WS-Security)** : C'est ici que l'on place les signatures numériques et le chiffrement. *Analogie : C'est le sceau de cire sur une lettre officielle de la Primature qui prouve que le contenu n'a pas été modifié.*
- **Routage et Transaction** : Permet de gérer les identifiants de transaction pour s'assurer qu'un virement à la **CBAO** n'est pas exécuté deux fois en cas de coupure réseau.
- **Attribut mustUnderstand** : Un paramètre unique à SOAP qui oblige le destinataire à lire et comprendre une instruction précise, sinon la transaction est annulée.

B. Le Body (Le Corps) : Le "Contenu du Bordereau"

- C'est la partie qui contient les données utiles (Payload).
- **Rôle** : Il transporte les paramètres d'entrée pour une requête (ex: montant, destinataire) ou les résultats pour une réponse.
- **Fault (Gestion d'erreurs)** : En cas de problème (ex: solde insuffisant), SOAP génère une structure spécifique appelée **SOAP Fault** à l'intérieur du Body. Cela permet au développeur de savoir exactement pourquoi l'action a échoué (Erreur client vs Erreur serveur).

WSDL : La "Constitution" du Service Web

- Le **WSDL (Web Services Description Language)** est le document le plus important pour l'interopérabilité. C'est un contrat XML qui décrit le "Quoi", le "Comment" et le "Où" du service.

A. La Théorie du "Contrat d'abord" (Contract-First)

- Au lieu d'écrire le code et d'espérer que ça marche, on définit d'abord le WSDL.
- **Analogie** : C'est comme le **Code des Marchés Publics**. Avant de poser la première pierre d'un pont, on publie un cahier des charges précis que tout le monde doit respecter à la lettre.
- **Composants du WSDL** :
 - **Types** : Les types de données (via XSD).
 - **Messages** : Ce qui entre et ce qui sort.
 - **PortType** : La liste des fonctions disponibles (ex: virement(), consultation()).
 - **Binding** : Comment le message est transporté (généralement sur HTTP).

B. Pourquoi l'interface rigide est vitale pour l'interopérabilité ?

- Imaginez un serveur central de la **Douane (GAINDE)** écrit en **Java** qui doit communiquer avec une application de logistique au Port d'Abidjan écrite en **.NET (C#)**.
- Grâce au WSDL, les deux systèmes génèrent automatiquement leur code à partir du même contrat.
- Il n'y a **aucun risque de mauvaise interprétation** : si le WSDL dit que la date doit être au format AAAA-MM-JJ, les deux systèmes s'y plieront, peu importe leur langage de programmation.

XSD (XML Schema Definition) : La "Police des Données"

- Le XSD est le langage utilisé à l'intérieur du WSDL pour définir la structure exacte des données.
- **Théorie du Typage Fort** : Contrairement au Web classique où tout est souvent du texte, le XSD impose des types. Un champ peut être défini comme `xs:decimal`, `xs:date`, ou même un type complexe personnalisé (ex: `TypeAdresseSenegal` avec région, département, commune).
- **Validation de documents** : C'est le processus qui vérifie que le message XML respecte le schéma.
 - *Analogie* : C'est comme le **contrôle technique** d'un véhicule à Dakar. Si les phares ne marchent pas (donnée manquante) ou si les pneus sont lisses (mauvais format), le véhicule ne passe pas. Le serveur rejette le message avant même qu'il n'atteigne la base de données de la banque.

Synthèse comparative : Pourquoi cette rigueur ?

Élément	Rôle	Équivalent dans la réalité sénégalaise
Enveloppe	Conteneur standard	Le pli recommandé de La Poste.
Header	Sécurité et Logistique	Le cachet de la police et les timbres fiscaux.
Body	Données métier	Le montant du chèque écrit à l'intérieur.
WSDL	Le contrat global	L'acte notarié qui définit les obligations.
XSD	La règle d'écriture	Le formulaire cerfa où chaque case a une taille fixe.

Étude de Cas : Le Choix de la BCEAO et de la CBAO

- Pourquoi, à l'ère du Web mobile et des APIs ultra-rapides, les institutions comme la **BCEAO** (Banque Centrale des États de l'Afrique de l'Ouest) imposent-elles encore **SOAP** pour les systèmes **SICA** (Système Interbancaire de Compensation Automatisé) et **STAR** (Système de Transfert Automatisé de Règlement) ?

A. La Rigueur du Contrat (Pas d'erreur d'interprétation)

- Dans un virement entre la CBAO et la SGBS, on ne peut pas se permettre une approximation.
- **Le WSDL comme Juge** : Si la BCEAO modifie une règle de virement, elle met à jour son fichier WSDL. Tous les serveurs des banques commerciales se synchronisent sur ce contrat. Si un message n'est pas 100% conforme au schéma **XSD**, il est rejeté par le "Gateway" avant même d'entrer dans le système comptable.
- **Analogie** : C'est le **chèque certifié**. Contrairement à un accord verbal, le chèque a un format physique strict. S'il manque une signature ou si le montant en lettres diffère du montant en chiffres, il est rejeté d'office.

B. La Sécurité Native (WS-Security)

Contrairement à REST qui repose souvent uniquement sur le cadenas du navigateur (HTTPS), SOAP possède ses propres blindages intégrés :

- **Signature Numérique** : Chaque ordre de virement est signé au niveau de l'enveloppe XML. Même si un pirate intercepte le message entre Dakar et Abidjan, il ne peut pas modifier le montant sans briser la signature.
- **Non-répudiation** : Une banque ne peut pas dire "Je n'ai pas envoyé cet ordre", car l'en-tête (Header) SOAP contient la preuve cryptographique de l'émetteur.

TP 2 : Création du Service "SunuBank" avec JAX-WS.

Projet Maven (Java) et mettre à jour le fichier pom.xml

- L'objectif est de simuler un service de **Consultation de Solde**. Nous allons utiliser **jakarta.jws** (Java API for XML Web Services), qui est le standard pour créer des services SOAP en Java.

Étape 1 : Définition de l'Interface (Le Contrat) . Créer un projet maven

- On utilise l'annotation `@WebService` pour dire à Java que cette classe sera exposée sur le réseau.

```

• package sn.sunubank;
import jakarta.jws.WebMethod;
import jakarta.jws.WebParam;
import jakarta.jws.WebService;
/**
 * Interface du Web Service SunuBank.
 * Cette interface définit les méthodes accessibles aux clients SOAP.
 */
@WebService(name = "SunuBankWS")
public interface BanqueService {
    /**
     * Consulte le solde d'un compte.
     * @param code numéro du compte à vérifier
     * @return solde du compte
     */
    @WebMethod(operationName = "getSolde")
    double consulterSolde(@WebParam(name = "numeroCompte") String code);
}

```

Étape 2 : Implémentation du Service (La Logique) et Étape 3 : Le Serveur (Publication de l'Endpoint)

Étape 2 : Implémentation du Service (La Logique)

- C'est ici que l'on simule la base de données de la banque.

```
package sn.sunubank;
import jakarta.jws.WebService;
/**
 * Implémentation du Web Service SunuBank.
 * Cette classe contient la logique métier du service bancaire.
 */
@WebService(endpointInterface = "sn.sunubank.BanqueService")
public class BanqueServiceImpl implements BanqueService {

    @Override
    public double consulterSolde(String code) {
        // Vérification de validité du numéro de compte
        if (code == null || code.isEmpty()) {
            System.out.println("Numéro de compte invalide !");
            return 0.0; // Retourne zéro pour éviter les erreurs
        }
        // Exemple de logique : comptes Orange Money commençant par 77
        if (code.startsWith("77")) {
            return 500000.0;
        }
        // Autres comptes
        return 150.0;
    }
}
```

Étape 3 : Le Serveur (Publication de l'Endpoint)

- On expose le service sur une URL précise pour qu'il soit accessible.

```
package sn.sunubank;
import jakarta.xml.ws.Endpoint;
/**
 * Classe principale pour démarrer le serveur SOAP.
 * Publie le service SunuBank sur l'URL spécifiée.
 */
public class SunuBankServer {
    public static void main(String[] args) {
        // URL où le service sera accessible
        String url = "http://localhost:9090/sunubank";
        // Publication du service
        Endpoint.publish(url, new BanqueServiceImpl());
        System.out.println("Banque SunuBank opérationnelle sur : "
+ url + "?wsdl");
    }
}
```

Étape 2 : TestSunuBankClient

```
• package sn.sunubank;

import jakarta.xml.ws.Service;
import java.net.URL;
import javax.xml.namespace.QName;

public class TestSunuBankClient {

    public static void main(String[] args) throws Exception {

        // URL du WSDL exposé par le serveur
        URL wsdlURL = new URL("http://localhost:9090/sunubank?wsdl");

        // QName : namespace et nom du service générés automatiquement par JAX-WS
        QName SERVICE_NAME = new QName("http://sunubank.sn/", "BanqueServiceImplService");

        // Création du service avec le QName correct
        Service service = Service.create(wsdlURL, SERVICE_NAME);

        // Obtention du port pour invoquer les méthodes
        BanqueService port = service.getPort(BanqueService.class);

        // Test de la méthode
        String numeroCompte = "771234567";
        double solde = port.consulterSolde(numeroCompte);

        System.out.println("Solde du compte " + numeroCompte + " = " + solde);
    }
}
```

Correction et Points d'Attention (Le "Checklist" de l'Ingénieur)

- Pour que votre TP soit validé niveau ingénieur, vérifiez les points suivants :
- **L'accès au WSDL** : Une fois le serveur lancé, ouvrez votre navigateur sur `http://localhost:8080/sunubank?wsdl`. Si vous voyez un fichier XML complexe avec des balises `<types>`, `<message>` et `<portType>`, votre service est bien déployé.
- **Test avec SoapUI ou Postman** : * Ne testez pas avec un navigateur (qui fait du GET).
 - Utilisez **SoapUI**. Importez l'URL du WSDL. Il va générer automatiquement l'enveloppe XML pour vous.
 - `curl http://localhost:9091/sunubank\?wsdl` ou `curl http://localhost:9091/sunubank?wsdl`
 - Postman : GET `http://localhost:9091/sunubank?wsdl`
- **Anatomie du message de test** : * Votre requête doit avoir un `<soapenv:Envelope>`.
 - Le corps (`<soapenv:Body>`) doit contenir la balise `<getSolde>`.
 - Le paramètre doit être `<numeroCompte>771234567</numeroCompte>`.

Pourquoi cette approche?

- En développant **SunuBank**, vous apprenez à construire des systèmes **interopérables**. Demain, votre service Java pourra être consommé par une application de micro-finance écrite en Python ou par le système central de la Poste (SIPE), car ils partagent le même contrat WSDL.

Module 3 :

Conception d'APIs RESTful (L'Approche Ressources)

Définition : REST, un Style, pas un Standard

- Contrairement à SOAP qui est un protocole avec un manuel d'utilisation rigide, **REST (Representational State Transfer)** est un **style d'architecture**.
- Il définit un ensemble de contraintes qui, si elles sont respectées, permettent de créer des services web performants, scalables et faciles à maintenir.
- L'idée centrale est la **Ressource** : tout objet que l'on veut manipuler (un étudiant de l'UCAD, une facture Senelec, une voiture de transport Dakar Dem Dikk) est une ressource accessible via une adresse unique (URL).

Analyse Détaillée des 6 Contraintes de Fielding

Pour qu'une API mérite l'appellation "RESTful", elle doit valider ces six piliers :

1. Séparation Client-Serveur

- **Le concept** : Les responsabilités sont strictement divisées. Le client s'occupe de l'affichage et de l'expérience utilisateur, tandis que le serveur gère le stockage et la logique métier.
- **L'avantage** : Vous pouvez changer toute votre application mobile (le client) sans toucher au serveur, et inversement.

2. Interface Uniforme (La clé de la simplicité)

- **Le concept** : Toutes les ressources sont manipulées de la même manière via les méthodes standard HTTP (GET, POST, PUT, DELETE).
- **Analogie** : C'est comme le **système des boutiques de quartier**. Que vous soyez à Saint-Louis ou à Ziguinchor, si vous demandez "Da nga am café ?=> est ce que tu as du café", le boutiquier comprend. Le protocole de communication est le même partout au Sénégal.

3. Système par Couches (Layered System)

- **Le concept** : Le client ne sait pas s'il parle directement au serveur final ou à un intermédiaire (serveur de sécurité, répartiteur de charge).
- **Analogie** : Quand vous payez une amende au **Trésor Public**, vous interagissez avec un guichetier. Vous ne savez pas si l'argent reste dans son tiroir ou s'il est envoyé immédiatement à la banque centrale. Cette couche intermédiaire protège le système central.

4. Mise en Cache (Cacheability)

- **Le concept** : Le serveur doit étiqueter ses réponses comme "cachables" ou non.
- **Impact Local** : Avec les connexions internet parfois instables ou coûteuses au Sénégal, le cache permet au téléphone de ne pas redemander une image (comme le logo de la banque) si elle n'a pas changé.

5. Code à la Demande (Facultatif)

- **Le concept** : Le serveur peut envoyer du code (JavaScript) pour étendre les fonctionnalités du client temporairement.

6. LA CONTRAINTE MAJEURE : Stateless (Absence d'état)

- C'est ici que réside la puissance de REST.

Focus Approfondi : Pourquoi le "Stateless" ?

- **Définition** : Dans une architecture Stateless, le serveur ne conserve aucune trace des interactions passées avec le client. Chaque requête est un "paquet" autonome qui contient tout : l'identité de l'utilisateur, ses permissions, et l'action demandée.

Pourquoi est-ce "Amnésique" ?

- Imaginez que vous développez une application pour les **Pharmacies de Garde à Dakar**.
- **Mode Stateful (Mauvais)** : Le client dit "Je veux une pharmacie". Le serveur répond "Ok, j'ai noté que tu es à Dakar". Le client dit "Donne-moi celle de garde". Le serveur doit se souvenir du contexte précédent (Dakar). Si le serveur redémarre entre-temps, il oublie tout et l'application plante.
- **Mode Stateless (REST)** : Le client dit à chaque fois : "Je suis Moussa (Token), je suis à Dakar, donne-moi les pharmacies de garde". Le serveur traite la demande et vous oublie immédiatement.

L'Analogie : "La Commande au Fast-Food" vs "Le Repas à Domicile"

- **Stateful (Repas à domicile)** : Votre maman sait ce que vous avez mangé hier, elle sait si vous avez faim. Elle a l'**état** de votre estomac en mémoire. Si une autre personne vient cuisiner, elle est perdue.
- **Stateless (Fast-food type KFC à Dakar ville)** : Le caissier ne vous connaît pas. Pour chaque commande, vous devez dire : "Menu 2 morceaux, sans piment, avec Coca". Même s'il y a 5 caissiers différents, n'importe lequel peut vous servir car vous donnez **toute l'information** à chaque commande.

Les Avantages pour un Ingénieur

- **Scalabilité (Évolutivité)** : Si votre service de pharmacies explose et que vous passez de 1 000 à 1 000 000 d'utilisateurs, vous ajoutez simplement des serveurs. Comme ils n'ont pas de mémoire à partager, la charge est répartie parfaitement.
- **Survie aux pannes** : Si un serveur brûle à cause d'un court-circuit, un autre prend le relais. Comme le client envoie toujours son "dossier complet" dans la requête, le nouveau serveur peut répondre sans interruption de service.
- **Visibilité** : Les outils de surveillance (monitoring) voient tout dans une seule requête, ce qui facilite le débogage.

Résumé

- Pour qu'un système soit REST, il doit traiter chaque requête comme une île isolée. Le serveur est un exécutant pur qui ne s'attache pas à l'utilisateur. C'est le prix à payer pour avoir un Web mondial, rapide et indestructible.
- Pour devenir un architecte d'APIs accompli, il faut comprendre que le nommage et la gestion des retours (codes d'état) constituent la **grammaire du Web**.
- Si vous ne respectez pas ces règles, votre API sera comme un livre écrit sans ponctuation : illisible pour les autres développeurs.

Modélisation de Ressources : La Philosophie du Nommage

En REST, on ne parle pas d'actions, on parle d'**Entités**. Une ressource est un concept abstrait ou concret que l'on veut exposer au monde.

Théorie du Nommage : Noms vs Verbes

- L'erreur fondamentale est de penser en "fonctions" (ex: calculerSalaire(), supprimerEtudiant()). En REST, on pense en "Données".
- **Le Verbe est dans la Méthode HTTP** : L'action est portée par le protocole (GET, POST, PUT, DELETE).
- **Le Nom est dans l'URL** : L'URL identifie la chose sur laquelle on agit.
 - **Invalid** : GET /getAllPharmacies (Verbe dans l'URL)
 - **Valide** : GET /pharmacies (L'action de lire est implicite par le GET)
- **Règle du Pluriel** : On utilise toujours le pluriel pour désigner une collection.
- /pharmacies : Représente l'ensemble des pharmacies de Dakar.
- /pharmacies/102 : Représente la pharmacie spécifique ayant l'ID 102.

Hiérarchie des Ressources (Le Chemin Logique)

- Les ressources sont souvent liées entre elles. L'URL doit refléter cette relation de possession (parent/enfant).
- **Concept** : /parent/{id-parent}/enfant/{id-enfant}
- **Exemple UCAD** : /universites/ucad/facultes/faseg/etudiants/202401
 - Ici, on comprend immédiatement que l'étudiant 202401 appartient à la FASEG, qui appartient à l'UCAD.

Analogie : Le "Kogne" (Quartier)

- Imaginez que vous cherchez une boutique à la Médina.
- Dakar / Médina / Rue-6 / Boutique-Moussa.
- Chaque segment de l'URL affine la recherche. Si vous enlevez Boutique-Moussa, vous avez la liste de toutes les boutiques de la Rue 6.

Logique des Codes d'État : La Sémantique de Réponse

- Le code HTTP est le premier élément qu'une application cliente (comme une app Android de livraison) regarde pour savoir si elle doit afficher une erreur ou une validation à l'utilisateur.

1. La Famille 2xx (Succès - "Tout va bien")

- **200 OK** : Utilisé pour un GET réussi ou un PUT (mise à jour).
- **201 Created** : Spécifique au POST. Le serveur confirme qu'il a créé la ressource (ex: "Nouvelle pharmacie enregistrée").
- **204 No Content** : Très utilisé pour le DELETE. L'action est faite, il n'y a plus rien à afficher.

2. La Famille 4xx (Erreurs Client - "C'est de ta faute")

- Le serveur est en bonne santé, mais ce que vous demandez est impossible ou interdit.
- **400 Bad Request** : La "poubelle" des erreurs de saisie. Vous avez envoyé du texte au lieu d'un prix en FCFA.
- **401 Unauthorized** : "Qui es-tu ?". Vous n'avez pas fourni de clé API ou de Token.
- **403 Forbidden** : "Je sais qui tu es, mais tu n'as pas le droit". Un pharmacien de Pikine essaie de modifier les stocks d'une pharmacie aux Almadies.
- **404 Not Found** : L'erreur la plus célèbre. Vous cherchez une ressource qui n'existe pas ou qui a été supprimée.

3. La Famille 5xx (Erreurs Serveur - "Je suis en panne")

- Ici, c'est le développeur ou l'administrateur système qui doit intervenir.
- **500 Internal Server Error** : Le code a rencontré une exception non gérée (ex: division par zéro dans le calcul des taxes).
- **503 Service Unavailable** : Souvent lié à une surcharge. *Analogie* : C'est comme le serveur des résultats du BAC le jour J ; trop de monde, le serveur "s'assoit".

Synthèse Appliquée : L'API "Pharmacie de Garde"

- Voici comment un ingénieur modélise proprement le système :

Objectif	URL	Code attendu
Lister les pharmacies	/pharmacies	200 OK
Filtrer celles de Dakar	/pharmacies?ville=dakar	200 OK
Ajouter une pharmacie	/pharmacies	201 Created
Modifier un numéro de téléphone	/pharmacies/12	200 OK
Supprimer (fermeture)	/pharmacies/12	204 No Content

Pourquoi utiliser PATCH plutôt que PUT ?

- PUT** : Vous remplacez toute la fiche de la pharmacie (Nom, Adresse, Ville, Tel). Si vous oubliez la Ville, elle devient vide.
- PATCH** : Vous ne modifiez **que** le téléphone. C'est plus léger et moins risqué.

TP 3 : Développement de l'API "Dakar-Pharma" avec Node.js & Express

Objectif du TP

- Créer un serveur capable de gérer les pharmacies de garde au Sénégal. Nous allons implémenter le **CRUD** complet :
- **Create** (Ajouter une pharmacie)
- **Read** (Lister/Filtrer les pharmacies de garde)
- **Update** (Modifier les informations)
- **Delete** (Supprimer une pharmacie fermée)

Configuration de l'environnement

- Avant de coder, préparez votre dossier de projet (dans votre terminal) :
- Bash
- `mkdir dakar-pharma-api`
- `cd dakar-pharma-api`
- `npm init -y`
- `npm install express`

Le Code Source Détaillé (server.js)

- Voici l'implémentation complète avec des commentaires pédagogiques pour chaque ligne.

- JavaScript

```
/**
 * IMPORTATION DES MODULES
 * Express est le framework web standard pour Node.js
 */
const express = require('express');
const app = express();
const PORT = 3000;
// Middleware pour permettre au serveur de lire le format JSON envoyé par le client
app.use(express.json());
/**
 * BASE DE DONNÉES SIMULÉE (En mémoire vive)
 * On utilise une liste d'objets pour représenter nos pharmacies à Dakar.
 */
let pharmacies = [
  { id: 1, nom: "Pharmacie de la Nation", ville: "Dakar", quartier: "Plateau", garde: true },
  { id: 2, nom: "Pharmacie Serigne Fallou", ville: "Touba", quartier: "Darou Marnane", garde: false },
  { id: 3, nom: "Pharmacie Cheikh Anta Diop", ville: "Dakar", quartier: "Fann", garde: true }
];
```

```
// =====
// 1. LIRE (READ) - Liste avec filtres (Query Params)
// URL testée : GET /pharmacies?ville=Dakar&garde=true
// =====

app.get('/pharmacies', (req, res) => {
  let resultat = [...pharmacies]; // On fait une copie de la liste
  // Récupération des filtres depuis l'URL (?ville=Dakar&garde=true)
  const { ville, garde } = req.query;
  if (ville) {
    resultat = resultat.filter(p => p.ville.toLowerCase() === ville.toLowerCase());
  }
  if (garde) {
    // req.query.garde est une chaîne de caractères, on la convertit en booléen
    const isGarde = (garde === 'true');
    resultat = resultat.filter(p => p.garde === isGarde);
  }
  // Réponse avec le code 200 OK
  res.status(200).json(resultat);
});
```

Le Code Source Détaillé (server.js) suite

```
// =====
// 2. CRÉER (CREATE) - Ajouter une nouvelle pharmacie
// URL testée : POST /pharmacies
// =====

app.post('/pharmacies', (req, res) => {
  const nouvellePharma = {
    id: pharmacies.length + 1,
    nom: req.body.nom,
    ville: req.body.ville,
    quartier: req.body.quartier,
    garde: req.body.garde || false
  };

  // Validation simple (Simulation XSD)
  if (!nouvellePharma.nom || !nouvellePharma.ville) {
    return res.status(400).json({ erreur: "Le nom et la ville sont obligatoires !" });
  }

  pharmacies.push(nouvellePharma);

  // 201 Created : Le code standard pour une création réussie
  res.status(201).json(nouvellePharma);
});
```

```
// =====
// 3. METTRE À JOUR (UPDATE) - Modifier une pharmacie
// URL testée : PUT /pharmacies/1
// =====

app.put('/pharmacies/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = pharmacies.findIndex(p => p.id === id);

  if (index !== -1) {
    pharmacies[index] = { ...pharmacies[index], ...req.body };
    res.status(200).json({ message: "Pharmacie mise à jour", data: pharmacies[index] });
  } else {
    // 404 Not Found : La pharmacie n'existe pas
    res.status(404).json({ erreur: "Pharmacie introuvable" });
  }
});
```

Le Code Source Détaillé (server.js) suite

```
// =====  
  
// 4. SUPPRIMER (DELETE) - Retirer une pharmacie  
  
// URL testée : DELETE /pharmacies/1  
  
// =====  
  
app.delete('/pharmacies/:id', (req, res) => {  
  const id = parseInt(req.params.id);  
  pharmacies = pharmacies.filter(p => p.id !== id);  
  // 204 No Content : Action réussie, rien à renvoyer  
  res.status(204).send();  
});
```

```
/**  
 * LANCEMENT DU SERVEUR  
 */  
  
app.listen(PORT, () => {  
  console.log(` Serveur Dakar-Pharma lancé sur http://localhost:${PORT} `);  
  console.log("Prêt pour les tests Postman !");  
});
```

Guide de test (Correction du TP) Lancer sur CMD : node server.js

Pour vérifier que votre API est "pro", utilisez **Postman** ou **cURL** :

Test A : Récupérer uniquement les pharmacies de garde à Dakar

- **Méthode** : GET
- **URL** : `http://localhost:3000/pharmacies?ville=Dakar&garde=true`
- **Résultat attendu** : Une liste JSON contenant uniquement la "Pharmacie de la Nation" et "Cheikh Anta Diop".

Test B : Ajouter une pharmacie à Pikine

- **Méthode** : POST
- **URL** : `http://localhost:3000/pharmacies`
- **Body (JSON)** :
- JSON

```
{  
  "nom": "Pharmacie de Pikine",  
  "ville": "Dakar",  
  "quartier": "Pikine Guinaw Rail",  
  "garde": true  
}
```

- **Résultat attendu** : Code 201 Created.

Test C : Supprimer une pharmacie inexistante

- **Méthode** : DELETE
- **URL** : `http://localhost:3000/pharmacies/999`
- **Résultat attendu** : Votre code doit être robuste. Ici, il renvoie 204 car l'élément n'est plus là (principe d'idempotence).

Analyse de l'ingénieur (Pourquoi ce code est RESTful ?)

- **Stateless** : Le serveur ne stocke pas de "session". Si vous redémarrez le serveur, la liste revient à son état initial. Chaque requête GET doit renvoyer ses filtres (?ville=...).
- **Noms vs Verbes** : Nos URLs sont /pharmacies (le nom de la ressource), pas /get-all-pharmas.
- **Codes d'état Sémantiques** : Nous utilisons 201 pour la création, 404 pour l'absence, et 400 pour les erreurs de saisie de l'utilisateur.

Module 4 : Documentation, Gouvernance et Sécurité

- **L'objectif** : Rendre ses services professionnels, sécurisés et utilisables par d'autres.

Théorie Approfondie

- Pour qu'une API ne soit pas juste un projet d'étudiant mais un service **industriel** (utilisable par exemple par une banque ou une administration publique au Sénégal), elle doit être prévisible, verrouillée et ouverte de manière contrôlée.

Théorie de l'API-First : L'Architecture par le Contrat

1. Le Concept de "Contrat de Service"

Dans l'approche traditionnelle, on code, puis on documente (si on a le temps). Dans l'approche **API-First**, le **contrat est la source de vérité**. On utilise le format **OpenAPI** (autrefois Swagger) pour définir :

- Chaque URL (Endpoints).
- Chaque paramètre attendu (Type, format, obligatoire ou non).
- Chaque réponse possible (Codes 200, 400, 401, 500).

2. Pourquoi gagne-t-on 30% de temps ?

- **Élimination du "re-travail"** : Les parties prenantes (clients, managers, développeurs front) valident le design *avant* le développement. Modifier un fichier texte (le contrat) prend 5 minutes ; modifier une base de données et 500 lignes de code prend 3 jours.
- **Génération de code** : À partir du contrat, on peut générer automatiquement le squelette du serveur et la documentation interactive.

Analogie : Le "Debeuss" (Plan de masse)

- Avant de construire un centre commercial à Diamniadio, l'architecte montre une maquette 3D aux investisseurs. On ne commence pas par acheter le fer et le ciment. Si l'investisseur veut changer la place de l'ascenseur, on le fait sur le plan. L'API-First, c'est cette maquette numérique qui garantit que le maçon (backend) et le décorateur (frontend) sont d'accord.

Sécurité des Échanges : JWT (JSON Web Token)

- Le **JWT** est l'outil ultime pour maintenir la contrainte **Stateless** de REST tout en assurant une sécurité maximale.

Pourquoi le JWT a remplacé les "Sessions" ?

- **Ancien modèle (Session)** : Le serveur garde une liste des gens connectés dans sa mémoire. Si le serveur redémarre (coupure de courant ou mise à jour), tout le monde est déconnecté.
- **Nouveau modèle (JWT)** : Le serveur donne un "ticket" sécurisé au client. Le client garde son ticket et le montre à chaque fois. Le serveur n'a plus besoin de mémoire.

L'Anatomie Profonde : Les 3 segments

- Un JWT ressemble à ceci : xxxxx.yyyyy.zzzzz
- **Le Header (Le tampon)** : Définit l'algorithme de cryptage (souvent HMAC SHA256). C'est le type de sceau utilisé.
- **Le Payload (Le message)** : C'est le contenu JSON. On y met les "Claims" (Revendications).
 - *Exemple* : {"sub": "pharmacien_123", "role": "ADMIN", "exp": 1716200000}. On sait qui c'est, son rôle, et quand le ticket expire.
- **La Signature (Le blindage)** : C'est la somme de (Header + Payload + **Clé Secrète du Serveur**).
 - **Important** : Si un pirate modifie le rôle de "USER" en "ADMIN" dans le Payload, la signature ne correspondra plus. Le serveur rejettera le jeton.

Analogie : Le Billet de Stade (Léopold Sédar Senghor)

Le JWT est comme un billet pour un match des Lions de la Teranga :

- Le billet contient la tribune, la rangée et l'heure (Payload).
- Le billet possède un hologramme ou un code-barres unique (Signature).
- Le contrôleur à la porte n'a pas besoin de vous connaître personnellement ou de vérifier votre nom sur une liste de 60 000 personnes. Il vérifie juste que l'hologramme est vrai. Si vous essayez de falsifier le numéro de siège, l'hologramme devient invalide.

CORS (Cross-Origin Resource Sharing)

- Le **CORS** est souvent la bête noire des développeurs, mais c'est une protection vitale du navigateur.

La "Same-Origin Policy" (SOP) : La Loi Fondamentale

- Par défaut, JavaScript sur le site mapharmacie.sn n'a pas le droit d'appeler l'API de ma-banque.sn. Pourquoi ? Pour éviter qu'un site malveillant ne vole vos données bancaires en utilisant votre session ouverte.

Comment fonctionne le CORS ?

C'est une négociation par Headers entre le navigateur et le serveur :

- **La Requête Preflight** : Pour les actions "dangereuses" (POST, DELETE, PUT), le navigateur envoie d'abord une requête OPTIONS. Il demande : "Est-ce que j'ai le droit de faire un DELETE depuis ce domaine ?"
- **La Réponse du Serveur** : Le serveur répond avec le Header Access-Control-Allow-Origin: <https://mon-front-app.com>.
- **L'Autorisation** : Si le domaine correspond, le navigateur envoie la vraie requête. Sinon, il bloque tout, même si le serveur est prêt à répondre.

Analogie : Le Protocole de Visite chez le Marabout

- Imaginez que vous voulez voir une autorité ou un chef religieux.
- Vous ne foncez pas directement dans sa chambre (**L'API**).
- Vous passez d'abord par le "Beukenek" (le serviteur/le garde du corps) qui est à la porte (**Le Navigateur/CORS**).
- Le Beukenek demande : "Qui t'envoie et d'où viens-tu ?".
- Si vous n'êtes pas sur la liste des personnes autorisées à venir de cette ville ou de cette famille, le Beukenek vous bloque à la porte, même si le marabout est à l'intérieur et disponible.

Résumé

Concept	Utilité Majeure	Ce qu'il faut retenir
API-First	Agilité et Clarté	On écrit le contrat Swagger/OpenAPI avant de coder.
JWT	Authentification	Pas de session sur le serveur. Tout est dans le Token signé.
CORS	Sécurité Navigateur	C'est le serveur qui choisit quels domaines peuvent l'appeler.

Le Rate Limiting : Protéger le Serveur contre l'Asphyxie

Définition

- Le **Rate Limiting** est une stratégie de gestion de trafic qui consiste à limiter le nombre de requêtes qu'un utilisateur (identifié par son adresse IP ou son Token JWT) peut effectuer auprès d'une API dans un intervalle de temps donné (ex: 100 requêtes par minute).
- Si l'utilisateur dépasse cette limite, le serveur rejette les requêtes supplémentaires avec un code d'erreur spécifique : **HTTP 429 Too Many Requests**.
- Le **Rate Limiting** (Limitation de débit) est la ceinture de sécurité de votre API. Sans elle, votre serveur est exposé à la saturation, qu'elle soit accidentelle ou malveillante. C'est un concept vital, surtout quand on déploie des services sur des infrastructures locales qui n'ont pas forcément la puissance de calcul des géants de la Silicon Valley.

Pourquoi est-ce crucial pour un serveur à Dakar ?

- Imaginez que vous hébergez votre API des pharmacies sur un petit serveur VPS ou une machine locale à Dakar. Vos ressources (CPU, RAM, Bande passante) sont limitées.

Éviter les attaques par Déni de Service (DoS/DDoS)

- Un pirate (ou un script mal réglé) pourrait envoyer 10 000 requêtes par seconde sur votre point d'entrée /pharmacies.
- **Conséquence** : Votre processeur monte à 100%, la mémoire s'emplit, et le serveur finit par "tomber" (crash). Plus aucun citoyen sénégalais ne peut alors accéder aux pharmacies de garde.
- **Le Rate Limiting** bloque ces requêtes à la porte avant qu'elles n'atteignent votre base de données.

La protection contre le "Web Scraping" sauvage

- Certaines personnes pourraient vouloir voler votre base de données complète des pharmacies pour la revendre. Ils utilisent des "bots" qui aspirent tout le contenu en quelques secondes. Limiter le débit ralentit ces robots et protège votre propriété intellectuelle.

L'Analogie : "Le Guichet de la SDE/Sen'Eau"

Imaginez une agence de la Sen'Eau à l'heure de pointe :

- S'il n'y a pas de vigile à la porte, 200 personnes se ruent en même temps vers le guichet. C'est la bousculade, personne n'est servi, et les vitres risquent de casser (**Crash du serveur**).
- **Le Rate Limiting, c'est le vigile et le système de tickets.** Le vigile ne laisse entrer que 5 personnes à la fois dans l'agence. Si vous arrivez et que c'est plein, il vous dit : "Attendez dehors, revenez dans 10 minutes" (**Code 429**). Cela garantit que ceux qui sont à l'intérieur sont servis sereinement et que le bâtiment reste debout.

Les Algorithmes de Limitation (Concept Avancé)

Pour implémenter cela, les ingénieurs utilisent souvent deux modèles :

- **Le Token Bucket (Le seau de jetons)** : Vous avez un seau de 10 jetons. Chaque requête consomme un jeton. Le seau se remplit d'un jeton toutes les secondes. Si le seau est vide, vous devez attendre.
- **Le Fixed Window (Fenêtre fixe)** : On autorise 60 requêtes entre 12h00 et 12h01. À 12h01, le compteur retombe à zéro.

Comment l'API prévient-elle l'utilisateur ?

Une API bien conçue envoie des **Headers** spécifiques pour aider le développeur (client) à ne pas se faire bloquer :

- X-RateLimit-Limit : Le nombre total autorisé (ex: 100).
- X-RateLimit-Remaining : Ce qu'il vous reste (ex: 45).
- X-RateLimit-Reset : Le temps restant avant que le compteur ne revienne à zéro.

Résumé

Concept	Action	Résultat
Normal	1 requête / sec	200 OK
Abusif	50 requêtes / sec	429 Too Many Requests
Objectif	Stabilité	Le serveur reste disponible pour tout le monde, tout le temps.

Auto- documentation avec Swagger UI

Le Concept : "Le Manuel qui s'écrit tout seul"

- Fini l'époque où l'on envoyait un fichier PDF de 50 pages (souvent pas à jour) aux autres développeurs. **Swagger (OpenAPI)** permet de générer une page Web interactive directement liée à votre code.
- **Pourquoi c'est révolutionnaire ?** Un développeur qui veut intégrer vos pharmacies de garde peut **tester** les appels (GET, POST) directement dans son navigateur sans écrire une seule ligne de code.

L'Analogie : "La Vitrine de la Boutique"

- Imaginez une boutique à Sandaga. Soit vous devez demander le prix de chaque article au vendeur (**Manuel PDF**), soit tous les articles sont exposés en vitrine avec une étiquette de prix et un bouton pour essayer (**Swagger**). C'est beaucoup plus efficace pour le client !

Implémentation (Aperçu Node.js)

- On utilise le module swagger-ui-express. On définit nos routes dans un fichier YAML ou via des commentaires JSDoc.
- JavaScript

// Exemple de commentaire Swagger pour la route GET /pharmacies

```
/**
```

```
* @swagger
```

```
* /pharmacies:
```

```
* get:
```

```
* summary: Récupère la liste des pharmacies de garde
```

```
* responses:
```

```
* 200:
```

```
* description: Liste des pharmacies trouvée
```

```
*/
```

Sécurisation : La Barrière de Sécurité (Middleware JWT)

Le Concept : "Le Badge de l'Ordre des Pharmaciens"

- Nous allons protéger les routes "sensibles" (comme la mise à jour des stocks ou l'ajout d'une pharmacie). Tout le monde peut **voir** les pharmacies (Public), mais seul le pharmacien authentifié peut **modifier** ses données (Privé).
- **Le mécanisme** : Le serveur vérifie la présence d'un **Token JWT** dans le Header Authorization de la requête.
- **L'Analogie** : "Le Checkpoint de la Gendarmerie"
- Sur la route de Diamniadio, tout le monde circule. Mais pour accéder à une zone de chantier restreinte, le gendarme (le **Middleware**) vous arrête et demande votre badge. Si vous n'avez pas le badge (le **Token**) ou s'il est périmé, il vous fait faire demi-tour (**Erreur 401 Unauthorized**).

TP 4 : Code de l'API Sécurisée et Documentée

Lancer sur CMD : node server.js

- Voici comment transformer notre serveur simple en un bastion sécurisé.
- Étape 1 : Installation des outils
- Bash
- `npm install jsonwebtoken swagger-ui-express swagger-jsdoc`
- Étape 2 : Le Code du Serveur avec Sécurité (server.js)
- JavaScript

```
const express = require('express');

const jwt = require('jsonwebtoken');

const swaggerUi = require('swagger-ui-express');
const swaggerJsdoc = require('swagger-jsdoc');

const app = express();

app.use(express.json());

const SECRET_KEY = "DAKAR_221_PROLE_KEY";

// Clé secrète pour signer les tokens
```

```
// --- CONFIGURATION SWAGGER ---

const swaggerOptions = {
  definition: {
    openapi: '3.0.0',
    info: { title: 'Dakar Pharma API', version: '1.0.0' },
  },
  apis: ['./server.js'],
};

const swaggerDocs = swaggerJsdoc(swaggerOptions);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));

// --- MIDDLEWARE DE SÉCURITÉ (Le Gendarme) ---

const verifierToken = (req, res, next) => {
  const bearerHeader = req.headers['authorization'];
```

TP 4 : Code de l'API Sécurisée et Documentée(suite)

```

if (typeof bearerHeader !== 'undefined') {
    const token = bearerHeader.split(' ')[1]; // On récupère le token après
    "Bearer "

    jwt.verify(token, SECRET_KEY, (err, authData) => {
        if (err) {
            res.status(403).json({ message: "Session expirée ou Token invalide" });
        } else {
            req.authData = authData; // On stocke les infos du pharmacien
            next(); // On laisse passer à la route suivante
        }
    });
} else {
    res.status(401).json({ message: "Accès refusé : Aucun badge (Token)
    trouvé" });
}
};

```

```

// --- ROUTES ---

// 1. LOGIN (Simulé) : Le pharmacien s'authentifie
app.post('/login', (req, res) => {

    // Dans la réalité, on vérifierait ici le mot de passe en base de données

    const user = { id: 1, username: 'pharmacie_plateau', role: 'PHARMACIEN' };

    // On génère le badge (Token) valable 1 heure
    jwt.sign({user}, SECRET_KEY, { expiresIn: '1h' }, (err, token) => {

        res.json({ token });

    });

});

// 2. ROUTE PROTÉGÉE : Mettre à jour le stock (Seulement avec Token)
app.put('/pharmacies/:id/stock', verifierToken, (req, res) => {

    // Si on arrive ici, c'est que le verifierToken a dit "OUI"

    res.json({

        message: "Stock mis à jour avec succès!",

        pharmacien: req.authData.user.username

    });

});

app.listen(3000, () => console.log("Lien Swagger : http://localhost:3000/api-docs"));

```

Guide de Correction (Comment tester ?)

- **Vérification Documentaire** : Allez sur <http://localhost:3000/api-docs>. Vous devez voir l'interface Swagger avec toutes vos routes listées proprement.
- **Test de la Sécurité** :
 - Essayez de faire un PUT sur `/pharmacies/1/stock` sans Token. Le serveur doit vous répondre 401 Unauthorized. **La barrière fonctionne.**
 - Faites un POST sur `/login`. Copiez le token reçu dans la réponse.
 - Refaites le PUT en ajoutant le header : `Authorization: Bearer [VOTRE_TOKEN]`.
 - Le serveur répond 200 OK. **Le pharmacien est passé !**

Pourquoi c'est "Pro" ?

- Grâce à ce système, votre API est prête à être utilisée par des partenaires (ex: une mutuelle de santé à Dakar). Ils lisent le **Swagger** pour comprendre comment vous appeler et utilisent le **JWT** pour prouver leur identité à chaque échange.

Module 5 : Intégration, Mashups et Webhooks

- **L'objectif** : Devenir un "orchestrateur" de services tiers pour créer de la valeur ajoutée.
- Le **Module 5** est l'étape où vous passez du statut de "développeur de site" à celui d'**ingénieur d'écosystème**.
- Dans le monde moderne, une application ne vit plus seule. Elle doit savoir parler aux autres, extraire de la valeur de sources diverses et réagir en temps réel aux événements extérieurs.

L'Architecture de Mashup : L'Intelligence par l'Assemblage

Définition

- Un **Mashup** est une application web hybride qui agrège dynamiquement des données, des services ou des fonctionnalités provenant de plusieurs sources disparates (via des API REST, SOAP, ou des flux RSS) pour créer un service nouveau et unique.
- Scientifiquement, le Mashup repose sur le principe de l'**abstraction de données** : votre application ne se soucie pas de *comment* les données sont stockées chez Google ou à la Senelec, elle ne consomme que leur interface (l'API).

Les Trois Niveaux de Mashup

- **Mashup au niveau Présentation (UI)** : On prend des morceaux d'interfaces (ex: une carte Google Maps et un widget météo) et on les colle sur la même page.
- **Mashup au niveau Données** : Le serveur récupère des données brutes (JSON/XML) de plusieurs APIs, les fusionne mathématiquement et renvoie un résultat combiné. C'est le plus puissant.
- **Mashup au niveau Processus** : On enchaîne des actions (ex: "Si l'API Météo annonce de la pluie à Dakar, alors appelle l'API SMS pour prévenir les agriculteurs").

Étude de Cas Approfondie : "Sunu-Energie-Map"

- Imaginons que nous voulions résoudre un problème réel à Dakar : **Corréler les coupures de courant avec les zones d'activité économique.**

A. La Collecte des "Ingrédients" (Sources de données)

- **API de Cartographie (ex: Mapbox ou Google Maps)** : Fournit les tuiles de la carte, les noms des rues de Dakar et la logique de zoom.
- **API de la Senelec (Données Réseau)** : Fournit l'état des postes haute tension (HT) et moyenne tension (MT). Imaginons un JSON :

```
{"poste": "Hann", "etat": "OFF", "cause": "Surcharge"}
```

.
- **API Places (Points d'Intérêt)** : Pour identifier où se trouvent les hôpitaux, les boulangeries et les banques dans la zone de Hann.

B. La Logique d'Orchestration (Le Code Middleware)

Votre serveur (en Node.js par exemple) va agir comme un **cerveau central** :

- **Étape 1 (Appel)** : Il demande à la Senelec quels secteurs sont coupés.
- **Étape 2 (Filtrage)** : Il demande à l'API Places les coordonnées de tous les hôpitaux de Dakar.
- **Étape 3 (Croisement Géographique)** : Le serveur calcule quels hôpitaux sont dans les zones "OFF" de la Senelec.
- **Étape 4 (Visualisation)** : Il renvoie au navigateur une carte où les hôpitaux en danger clignotent en rouge.

Analogie : "Le Guide du Grand Magal de Touba"

Imaginez une application mobile pour les pèlerins du Magal :

- **Ingrédient 1** : L'API de la Gendarmerie ou de Google Traffic pour voir les bouchons sur l'autoroute à péage et la route de Diourbel.
- **Ingrédient 2** : L'API d'une association locale qui liste les "Dahiras" offrant des repas gratuits (Berndèl).
- **Ingrédient 3** : L'API de localisation des forages pour trouver de l'eau.
- **Le Mashup** : Le pèlerin voit une seule carte. Il sait qu'à 2km de lui, il y a un bouchon, mais qu'à 500m, il y a un Dahira qui distribue du café et de l'eau.
- **L'ingénieur n'a créé ni la route, ni le café, ni l'eau. Il a créé l'intelligence** qui permet de les trouver au bon moment. C'est cela, la force du Mashup.

Les Défis Techniques du Mashup

- **Latence** : Si l'API A met 2 secondes à répondre et l'API B met 3 secondes, votre application mettra au moins 3 secondes. Il faut gérer l'asynchronisme.
- **Dépendance** : Si Google Maps change ses tarifs ou si la Senelec coupe son API, votre application "meurt". C'est le risque de la dépendance envers des tiers.
- **Formatage** : L'API A peut envoyer du XML et l'API B du JSON. Votre serveur doit savoir "traduire" les deux.

Résumé

Concept	Définition Courte	Application au Sénégal
Consommateur	Votre application Mashup	L'app "Sunu-Energie"
Fournisseur	L'API tierce	Google, Senelec, Orange Money
Agrégation	Action de mélanger les données	Croiser les pannes et les adresses
Valeur Ajoutée	Ce que vous créez de nouveau	L'alerte pour les hôpitaux

Webhooks vs Polling : L'Efficacité Énergétique et Réseau

Le Polling (L'Interrogation Cyclique) : Le Modèle "Bruit de Fond"

• A. Le Concept Technique : "Pull-based"

- Dans un système de **Polling**, le client est l'acteur principal. C'est lui qui prend l'initiative de demander une mise à jour à intervalles réguliers.
- **Short Polling** : On demande toutes les 2 secondes. Si la réponse est vide, on ferme la connexion et on recommence 2 secondes après.
- **Long Polling** : Le serveur garde la connexion "ouverte" un peu plus longtemps (ex: 20 secondes) tant qu'il n'y a pas de nouvelle donnée. C'est un peu moins lourd, mais cela bloque des ressources sur le serveur.

• B. Les Défis du Polling

- Au Sénégal, où le coût de la donnée mobile peut être un frein et où les serveurs locaux (souvent de petites instances) doivent économiser leurs ressources :
- **Gaspillage de Batterie** : Si votre application mobile fait du polling, le processeur du téléphone ne dort jamais. La batterie se vide en quelques heures.
- **Saturation des Sockets** : Chaque requête HTTP ouvre une "porte" (socket) sur le serveur. Si 10 000 utilisateurs font du polling en même temps, le serveur n'a plus de portes disponibles pour les nouveaux clients, même s'il ne fait rien d'utile.

Le Webhook (Le Rappel HTTP) : Le Modèle "Event-Driven"

A. Le Concept Technique : "Push-based"

Le **Webhook** est une inversion totale de la responsabilité. C'est ce qu'on appelle une **Architecture Pilotée par les Événements (Event-Driven Architecture)**.

- **Inscription** : Le client (votre site e-commerce) donne une URL de "rappel" (Callback URL) au serveur tiers (ex: Wave).
- **Attente Passive** : Le client ne fait plus rien. Il "dort".
- **Déclenchement (Trigger)** : Un événement se produit sur le serveur tiers (le client a validé son code secret).
- **Notification** : Le serveur tiers envoie immédiatement une requête POST vers l'URL de rappel avec les détails en JSON.

B. Pourquoi c'est "Magique" ?

- C'est la différence entre **chercher** une information et **être informé**. Le serveur devient proactif.

Analogie : "Le Livraison à Domicile vs Aller au Marché"

- **Polling (Aller au Marché) :** Vous voulez du poisson frais de Soumbédioune. Toutes les heures, vous prenez un taxi pour aller voir si les pirogues sont arrivées. Vous dépensez du temps, de l'argent de transport (bande passante) et souvent, vous revenez bredouille. C'est le **Polling**.
- **Webhook (Le Livreur Tiak-Tiak) :** Vous donnez votre numéro de téléphone et votre adresse à un pêcheur. Vous restez tranquillement chez vous à faire autre chose. Dès que la pirogue touche le sable, le pêcheur vous appelle ou envoie un livreur directement à votre porte. Vous n'avez fait aucun effort de déplacement inutile. C'est le **Webhook**.

La Sécurité des Webhooks : Le Concept de Signature

- Puisque votre URL de Webhook est "publique" (pour que Wave puisse vous appeler), un pirate pourrait essayer d'envoyer un faux message JSON :
`{"status": "SUCCESS", "amount": 1000000}` pour vous faire croire qu'il a payé.

Comment se protéger ? (La Gouvernance)

- **Secret Partagé** : Le fournisseur (Wave) et vous partagez une clé secrète.
- **HMAC (Hash-based Message Authentication Code)** : Wave envoie un Header spécial (ex: X-Wave-Signature). Ce Header contient le hash du message signé avec la clé secrète.
- **Vérification** : Votre serveur recalcule le hash avec la clé secrète. Si le résultat est identique, le message est authentique. C'est la **Confiance par la Preuve**.

5. Comparaison Synthétique

Critère	Polling	Webhook
Philosophie	"Est-ce que c'est prêt ?"	"C'est prêt, tiens !"
Sens de la donnée	Client $\leftarrow$ Serveur (sur demande)	Serveur $\rightarrow$ Client (automatique)
Consommation	Élevée et constante	Nulle entre deux événements
Idéal pour...	Vérifier la météo (peu de changements)	Paiements, SMS, Alertes de sécurité

TP 5 : Simulation d'un Webhook de Paiement Bash

mkdir TP5Paiement - cd TP5Paiement - npm init -y - npm install express

Imaginons que vous recevez une notification de paiement de PayTech ou Wave.

JavaScript

```
const express = require('express');

const crypto = require('crypto'); // Pour la sécurité (Signature)

const app = express();

app.use(express.json());

const SECRET_WEBHOOK = "CLE_SECURE_SENEGAL_2024"; // Clé partagée avec le fournisseur

// L'URL que vous donnez au fournisseur de paiement

app.post('/webhook/paiement-terme', (req, res) => {

  // 1. Récupérer la signature envoyée dans les headers

  const signature = req.headers['x-pay-signature'];

  // 2. Vérifier l'authenticité (Sécurité)

  const hash = crypto.createHmac('sha256', SECRET_WEBHOOK)

    .update(JSON.stringify(req.body))

    .digest('hex');
```

```
if (hash === signature) {

  // Le paiement est RÉEL et validé par le partenaire

  const { montant, client, reference } = req.body;

  console.log(` Paiement de ${montant} FCFA reçu de ${client} (Ref: ${reference}) `);

  // On peut maintenant débloquent la commande dans notre base de données

  res.status(200).send('Notification reçue et validée !');

} else {

  // Tentative de fraude détectée

  console.error("ALERTE : Signature invalide !");

  res.status(401).send('Fraude détectée');

}

});

app.listen(4000, () => console.log("En attente de notifications sur le port 4000..."));
```

TP 5 : Simulation d'un Webhook de Paiement Bash: node paiement.js

Étape 1 : Configurer la requête (Body)

- Ouvrez Postman et créez une nouvelle requête **POST**.
- Entrez l'URL : `http://localhost:4000/webhook/paiement-termine`.
- Allez dans l'onglet **Body**, cochez **raw** et sélectionnez **JSON** dans la liste déroulante.
- Collez ce JSON exactement (sans espaces superflus après les accolades) :
- JSON

```
{  
  "montant": 5000,  
  "client": "Fatou Ndiaye",  
  "reference": "CMD-2024-X"  
}
```

Étape 2 : Générer la signature (Pre-request Script)

- Pour que votre serveur valide la requête, le header `x-pay-signature` doit correspondre au hash du JSON ci-dessus avec votre clé `CLE_SECURE_SENEGAL_2024`.
- Allez dans l'onglet **Pre-request Script** de Postman.
- Collez le code suivant (Postman va calculer le hash automatiquement avant d'envoyer) :
- JavaScript

```
// 1. Définir la clé secrète (la même que dans votre code Node.js)
```

```
const secret = "CLE_SECURE_SENEGAL_2024";
```

```
// 2. Récupérer le corps de la requête (le JSON)
```

```
const body = pm.request.body.raw;
```

```
// 3. Calculer le Hash HMAC SHA256
```

```
const hash = CryptoJS.HmacSHA256(body, secret).toString(CryptoJS.enc.Hex);
```

```
// 4. Ajouter automatiquement le header à la requête
```

```
pm.request.headers.add({
```

```
  key: 'x-pay-signature',
```

```
  value: hash
```

```
});
```

```
console.log("Signature générée : " + hash);
```

TP 5 : Simulation d'un Webhook de Paiement Bash

Étape 3 : Envoyer et Vérifier

- Cliquez sur **Send**.
- **Dans Postman** : Vous devriez recevoir un statut 200 OK avec le message "Notification reçue et validée !".
- **Dans votre terminal (Node.js)** : Vous devriez voir s'afficher :
-  **SUCCÈS** : Paiement de 5000 FCFA reçu de Fatou Ndiaye.

Étape 4 : Tester la sécurité (Le test du "Hacker")

- C'est ici que vous comprenez l'utilité du code :
- Modifiez simplement le montant dans le JSON de Postman (ex: mettez 10 au lieu de 5000).
- **Désactivez** temporairement le script dans "Pre-request Script" (ou changez une lettre dans la clé secrète du script).
- Cliquez sur **Send**.
- **Résultat** : Votre serveur Node.js affichera  **ALERTE** : Signature invalide ! et renverra une erreur 401 Unauthorized.
- Cela prouve que même si quelqu'un connaît votre URL de Webhook, il ne peut pas valider de faux paiements sans votre clé secrète.

Gérer les doublons (ou l'idempotence)

- Gérer les **doublons** (ou l'idempotence) est l'aspect le plus critique d'un Webhook en production.
- **Le problème** : Parfois, le serveur du fournisseur de paiement ne reçoit pas votre "OK" (à cause d'une micro-coupure réseau) et renvoie la même notification 2 minutes plus tard.
- Si vous ne vérifiez rien, vous risquez de livrer le produit deux fois ou de créditer le compte du client à double.

TP6 : La solution : Le registre des transactions: node paiement.js

- Pour éviter cela, on doit enregistrer chaque reference de transaction traitée avec succès.
- Voici le code mis à jour pour bloquer les doublons :
- JavaScript

```
const express = require('express');
const crypto = require('crypto');
const app = express();
app.use(express.json({
  verify: (req, res, buf) => { req.rawBody = buf.toString(); }
}));
const SECRET_WEBHOOK = "CLE_SECURE_SENEGAL_2024";
// --- SIMULATION D'UNE BASE DE DONNÉES ---
// En vrai, on utiliserait MySQL ou MongoDB
const transactionsTraitees = new Set();
app.post('/webhook/paiement-termine', (req, res) => {
  const signatureFournisseur = req.headers['x-pay-signature'];
  const hashLocal = crypto.createHmac('sha256', SECRET_WEBHOOK)
    .update(req.rawBody)
    .digest('hex');
```

```
if (hashLocal !== signatureFournisseur) {
  return res.status(401).send('Signature invalide');
}

const { reference, montant } = req.body;
// --- VÉRIFICATION DU DOUBLON ---
if (transactionsTraitees.has(reference)) {
  console.log(` ⚠ DOUBLON DÉTECTÉ : La transaction ${reference} a déjà été traitée. `);
  // IMPORTANT : On renvoie quand même 200 OK au fournisseur
  // pour qu'il arrête d'envoyer cette notification.
  return res.status(200).send('Déjà traité');
}
// --- TRAITEMENT NORMAL ---
console.log(` ✅ PREMIER TRAITEMENT : Validation de ${montant} FCFA (Ref: ${reference}) `);
// On ajoute la référence à notre "liste noire" de transactions terminées
transactionsTraitees.add(reference);
res.status(200).send('Paiement validé avec succès');
});
app.listen(4000, () => console.log("Serveur anti-doublon actif sur le port 4000"));
```

Test avec Postman (Le cas du doublon)

- Pour tester cela, suivez ces étapes :
- **Lancez votre serveur** avec le nouveau code ci-dessus.
- **Premier envoi** : Allez sur Postman et cliquez sur **Send** (gardez les mêmes réglages que tout à l'heure).
 - *Réponse serveur* : Paiement validé avec succès.
 - *Console* :  PREMIER TRAITEMENT...
- **Deuxième envoi** : Cliquez à nouveau sur **Send** immédiatement (sans rien changer).
 - *Réponse serveur* : Déjà traité.
 - *Console* :  DOUBLON DÉTECTÉ...
- **Pourquoi renvoyer un code 200 même pour un doublon ?**
- C'est la règle d'or des Webhooks. Si vous renvoyez une erreur (400 ou 500), le système du fournisseur pensera que votre serveur est en panne et il continuera de vous "harcéler" avec la même notification toutes les heures pendant 2 jours. En répondant 200, vous lui dites : *"C'est bon, j'ai l'info, tu peux arrêter !"*.

À retenir pour la mise en production :

- **Base de données** : Dans un vrai projet, utilisez une colonne unique dans votre base de données SQL pour la référence de transaction.
- **Logs** : Gardez toujours une trace (un fichier log) des signatures reçues pour pouvoir prouver un paiement en cas de litige avec le client.

Conclusion

- Vous avez maintenant toutes les clés : du protocole **SOAP** rigide pour les banques, au style **REST** flexible pour le web, jusqu'à l'**Orchestration** et aux **Webhooks** pour les systèmes en temps réel.
- **Félicitations Ingénieur !**
- Dans un environnement comme le Sénégal, où la connectivité peut varier entre la fibre optique de Dakar-Plateau et une connexion 3G instable à l'intérieur du pays, la **gestion des erreurs réseau** n'est pas une option : c'est une question de survie pour votre application.
- Un ingénieur réseau doit concevoir son système en partant du principe que **le réseau va échouer**. Voici la théorie profonde pour rendre vos services Web "résilients".

La Résilience Réseau : Retry, Timeout et Idempotence

La Théorie du Timeout (Délai d'attente)

Définition

- Le **Timeout** est une contrainte de temps imposée à une requête. Si le serveur ne répond pas dans le délai imparti, le client coupe la connexion pour éviter de bloquer ses propres ressources (mémoire, threads) indéfiniment.

Pourquoi est-ce vital ?

- Sans Timeout, si le serveur de la banque (ex: CBAO) est surchargé ou si le câble sous-marin est coupé, votre application mobile va "freezer" (geler). L'utilisateur verra un écran blanc infini.
- **Connection Timeout** : Temps max pour établir le contact avec le serveur.
- **Read Timeout** : Temps max pour recevoir les données une fois connecté.

La Théorie du "Retry" (La Relance)

Le Concept : "Réessayer, mais intelligemment"

- Lorsqu'une erreur survient (code 503, timeout, perte de signal), l'application peut décider de renvoyer la requête automatiquement. Mais attention : faire des "Retry" sans stratégie peut achever un serveur déjà fatigué (effet de bord).

Stratégie : L'Exponential Backoff (Retrait Exponentiel)

- Au lieu de réessayer toutes les secondes, on augmente le temps d'attente entre chaque essai.
- Essai 1 : après 1 seconde.
- Essai 2 : après 2 secondes.
- Essai 3 : après 4 secondes.
- Essai 4 : après 8 secondes.

Analogie : "L'Appel au Cousin au Village"

Imaginez que vous essayez d'appeler un parent dans une zone où le réseau Orange/Free passe mal :

- **Mauvaise méthode** : Vous saturez le bouton "Appeler" toutes les 2 secondes. Vous videz votre batterie et saturez l'antenne relais.
- **Méthode Ingénieur (Retry)** : Vous essayez une fois. Ça ne passe pas ? Vous attendez 1 minute. Toujours pas ? Vous attendez 5 minutes. Puis 15 minutes. Vous économisez vos ressources et laissez au réseau le temps de se rétablir.

Le Concept d'Idempotence (Indispensable pour le Retry)

C'est le concept le plus important en finance (Wave, Orange Money).

- **Définition** : Une opération est **idempotente** si le fait de l'exécuter plusieurs fois a le même effet que de l'exécuter une seule fois.
- **Le Danger** : Si vous faites un POST /virement et que la connexion coupe juste après, vous ne savez pas si l'argent est parti. Si vous faites un "Retry" sans précaution, vous risquez de débiter le client **deux fois**.
- **La Solution** : On utilise une **Clé d'Idempotence** (un ID unique envoyé dans le Header). Le serveur vérifie : "Ah, j'ai déjà reçu cet ID de transaction, je ne traite pas à nouveau, je renvoie juste le reçu précédent".

Étude de Cas : Paiement de Facture Senelec via Mobile Money

Imaginons un utilisateur à Guédiawaye qui paie sa facture. Le réseau 4G saute au milieu de la transaction.

- **Phase 1 (Échec)** : L'app envoie le paiement. Le serveur Senelec reçoit l'argent mais le réseau coupe avant que l'app ne reçoive le "OK".
- **Phase 2 (Timeout)** : L'app détecte que le délai de 10 secondes est dépassé. Elle ne dit pas "Échec", elle dit "Vérification en cours...".
- **Phase 3 (Retry avec Idempotence)** : L'app réessaie l'envoi avec le même ID_TRANSACTION_789.
- **Phase 4 (Serveur)** : Le serveur voit ID_TRANSACTION_789. Il répond : "C'est déjà payé, voici ton reçu".
- **Résultat** : Le client est content, pas de double débit malgré la mauvaise connexion.

Synthèse

Problème	Solution Technique	Résultat
Lenteur Réseau	Timeout court	Éviter le gel de l'application.
Coupure Brève	Retry (Relance)	Expérience utilisateur fluide.
Surcharge Serveur	Exponential Backoff	Ne pas achever un serveur en panne.
Double Paiement	Idempotence	Sécurité financière absolue.

TP 7 : Simulation d'une fonction de "Retry" robuste : node retry.js

- Ce TP touche à un concept crucial en informatique : la **résilience**. En réseau (surtout avec des connexions parfois instables comme on peut en avoir partout), on ne peut pas supposer qu'une requête marchera du premier coup.
- Le concept utilisé ici est l'**Exponential Backoff** (attente exponentielle) : on attend de plus en plus longtemps entre chaque essai pour laisser au réseau le temps de "souffler".
- Voici comment coder une fonction qui n'abandonne pas au premier problème de connexion :

```
• JavaScript
async function appelerApiAvecRetry(url, maxRetries = 3) {
  let attente = 1000; // 1 seconde au début
  for (let i = 0; i < maxRetries; i++) {
    try {
      console.log(` Tentative ${i + 1} pour ${url}... `);
      const response = await fetch(url, { timeout: 5000 }); // Timeout de 5s
      return await response.json();
    } catch (err) {
      if (i === maxRetries - 1) throw err; // Si c'est le dernier essai, on abandonne

      console.log(` Échec. Réessai dans ${attente/1000} secondes... `);
      await new Promise(res => setTimeout(res, attente));
      attente *= 2; // On double le temps d'attente (Exponentiel)
    }
  }
}
```

TP 7 : Tests = node retry.js

2. Comment le lancer et le tester ?

- Comme ce code utilise fetch (disponible nativement dans Node.js 18+ et dans les navigateurs), vous avez deux options simples :

Option A : Dans votre navigateur (Le plus rapide)

- Ouvrez une page blanche sur Chrome ou Firefox.
- Faites un clic droit -> **Inspecter** -> onglet **Console**.
- Copiez et collez tout le code ci-dessus et appuyez sur **Entrée**.
- **Observez** : Vous verrez les messages apparaître avec un délai qui double à chaque fois.

Option B : Avec Node.js (Si version 18 ou plus)

- Enregistrez le code dans un fichier nommé retry.js.
- Dans votre terminal, tapez : node retry.js.

3. Pourquoi ce code est "Robuste" ?

- **La boucle for** : Elle limite strictement le nombre de tentatives. Sans cela, vous pourriez créer une boucle infinie qui fait planter votre serveur ou celui de votre partenaire (attaque DDOS involontaire).
- **attente *= 2** : C'est la clé de la courtoisie réseau. Si le serveur en face est surchargé, le bombarder de requêtes toutes les millisecondes va l'achever. En doublant l'attente, on lui donne une chance de redémarrer.
- **Promise + setTimeout** : C'est la manière moderne de "mettre en pause" une fonction async sans bloquer tout le reste de votre programme.

Architecture du Système

- **Le Webhook** : Une route POST qui attend que Wave dise "C'est payé !".
- **La Sécurité** : Vérification d'une signature pour éviter les faux paiements (piratage).
- **L'Action** : Envoi automatique d'un SMS via une API tierce (type Orange/InTouch).

TP8 Le Code Complet (app.js) : Voici le serveur Node.js détaillé et robuste.

JavaScript

```
const express = require('express');

const crypto = require('crypto'); // Pour la sécurité des signatures

const axios = require('axios'); // Pour appeler l'API SMS d'Orange/InTouch

const app = express();

app.use(express.json());

const PORT = 4000;

/** * CONFIGURATION SÉCURITÉ & API

* Dans un vrai projet, ces clés sont dans un fichier .env (secret)

*/

const WAVE_WEBHOOK_SECRET = "CLE_SECRET_WAVE_SN_2026";

const ORANGE_SMS_TOKEN = "TOKEN_API_ORANGE_SECRET";

// --- BASE DE DONNÉES SIMULÉE ---

let inscriptions = [

  { id: "CMD-101", nom: "Moussa Diop", tel: "771234567", status: "EN_ATTENTE" }

];
```

```
/**

* 1. LE WEBHOOK DE PAIEMENT (Simulation Wave/PayTech)

* C'est l'URL que vous configurez sur votre dashboard Wave.

*/

app.post('/webhooks/paiement', async (req, res) => {

  console.log("🔔 Notification de paiement reçue...");

  // A. SÉCURITÉ : Vérification de la signature HMAC

  // Wave envoie une signature pour prouver que c'est bien lui qui appelle

  const signatureRecue = req.headers['x-wave-signature'];

  const payload = JSON.stringify(req.body);

  const signatureAttendue = crypto

    .createHmac('sha256', WAVE_WEBHOOK_SECRET)

    .update(payload)

    .digest('hex');

  if (signatureRecue !== signatureAttendue) {

    console.error("❌ Alerte Sécurité : Signature invalide ! Tentative de fraude.");

    return res.status(401).send("Non autorisé");

  }

});
```

Le Code Complet (app.js) : Voici le serveur Node.js détaillé et robuste (suite)

// B. TRAITEMENT DE LA DONNÉE

```
const { id_commande, status, montant } = req.body;

if (status === "SUCCESS") {
  console.log(`✅ Paiement de ${montant} FCFA validé pour la commande ${id_commande}`);

  // Mise à jour du status dans notre "Base de données"

  const index = inscriptions.findIndex(i => i.id === id_commande);

  if (index !== -1) {
    inscriptions[index].status = "VALIDÉ";

    const clientTel = inscriptions[index].tel;

    // C. ACTION AUTOMATIQUE : ENVOI DU SMS

    await envoyerSmsConfirmation(clientTel, inscriptions[index].nom);
  }
}

// On répond toujours 200 OK à un Webhook pour dire qu'on a bien reçu le message

res.status(200).send("Reçu !");

});
```

/** * 2. FONCTION D'ENVOI SMS (Mashup avec API Orange/InTouch) */

```
async function envoyerSmsConfirmation(telephone, nom) {

  console.log(`📠 Préparation du SMS pour ${telephone}...`);

  const message = `Félicitations ${nom}, votre inscription sur SunuBank est validée !`;

  try {

    // Simulation d'un appel à l'API Orange SMS

    // Dans la réalité : await axios.post('https://api.orange.com/sms/v1/send', ...)

    console.log(`🚀 SMS ENVOYÉ via Passerelle Orange : [${message}]`);

    return true;

  } catch (error) {

    console.error("❌ Échec de l'envoi SMS :", error.message);

    // Ici, on pourrait implémenter une logique de "Retry" (Relance)

  }
}

app.listen(PORT, () => {

  console.log(`🚀 Serveur Marchand prêt sur http://localhost:${PORT}`);

  console.log(`📌 URL Webhook à donner à Wave : http://localhost:${PORT}/webhooks/paiement`);

});
```

Tester le code : node wave.js

2. Guide de Test Ultra-Détaillé avec Postman

- Pour tester ce code, vous devez "mentir" à votre serveur en lui faisant croire que vous êtes Wave. Mais comme votre serveur vérifie la signature, vous devez d'abord calculer la bonne signature.

Étape A : Préparer le Body (JSON)

- Lancez votre serveur (node le_nom_de_votre_fichier.js).
- Ouvrez Postman et créez une requête **POST** vers `http://localhost:4000/webhooks/paiement`.
- Allez dans **Body** > **raw** > **JSON** et collez exactement ceci (attention aux espaces !) :

```

{
  "id_commande": "CMD-101",
  "status": "SUCCESS",
  "montant": 2500
}

```

Étape B : Calculer la Signature (Pre-request Script)

Pour que le serveur accepte votre requête Postman, nous allons utiliser l'onglet **Pre-request Script** pour générer le header x-wave-signature automatiquement.

Cliquez sur l'onglet **Pre-request Script**.

Collez ce script :

JavaScript

// La clé doit être identique à celle du serveur

```
const secret = "CLE_SECRET_WAVE_SN_2026";
```

// Récupérer le corps brut

```
const body = pm.request.body.raw;
```

// Calculer le HMAC SHA256

```
const hash = CryptoJS.HmacSHA256(body, secret).toString(CryptoJS.enc.Hex);
```

// Ajouter le header requis par notre code

```
pm.request.headers.add({
```

```
  key: 'x-wave-signature',
```

```
  value: hash
```

```
});
```

```
console.log("Signature calculée pour le test : " + hash);
```

Tester le code

- **Étape C : Lancer le test**
- Appuyez sur **Send**.
- **Vérifiez votre console Node.js :**
 - Elle doit afficher : 🔔 Notification de paiement reçue...
 - Puis : 💰 Paiement de 2500 FCFA confirmé...
 - Et enfin : 🚀 SMS ENVOYÉ avec succès...

Pourquoi c'est une architecture "pro" ?

- **Sûreté** : Personne ne peut déclencher l'envoi de SMS gratuits en appelant votre URL, car ils n'ont pas la CLE_SECRET_WAVE_SN_2026.
- **Automatisation** : Le client paie sur son téléphone (Wave), et 2 secondes plus tard, il reçoit son SMS de confirmation sans aucune intervention humaine.
- **Traçabilité** : Vous avez des logs clairs pour chaque étape (Paiement -> Base de données -> SMS).

Explications Détaillées des Concepts

La Signature HMAC (Le "Cachet de la Poste")

- Dans le code, nous ne faisons pas confiance aveuglément au JSON reçu.
- **Pourquoi ?** N'importe qui connaissant votre URL pourrait envoyer {"status": "SUCCESS"} pour obtenir votre service gratuitement.
- **Le Concept :** Wave calcule une empreinte numérique (signature) avec une clé que vous êtes les seuls à connaître. Si quelqu'un change un seul chiffre dans le montant, l'empreinte ne correspondra plus.

Le Mashup de Services (Orchestration)

Votre serveur devient un chef d'orchestre :

- Il écoute **Wave** (Entrée).
- Il met à jour sa propre **Base de Données** (Interne).
- Il commande l'API d'**Orange** (Sortie).
- C'est la définition même d'un service Web moderne : faire dialoguer des systèmes qui ne se connaissent pas.

Conclusion de la Formation

Vous avez maintenant parcouru tout le cycle :

- **Module 1 & 2** : Comprendre les bases et la rigueur de **SOAP** (Banque Centrale).
- **Module 3** : Modéliser des ressources propres avec **REST** (Pharmacies).
- **Module 4** : Verrouiller vos accès avec **JWT** et documenter avec **Swagger**.
- **Module 5** : Réagir aux événements en temps réel avec les **Webhooks** et envoyer des **SMS**.
- **Bravo !** Vous avez toutes les compétences pour concevoir la prochaine grande Fintech Africaine . SN



Fin

Dr Papa Samour DIOP
Enseignant chercheur en Informatique